

Optimization Perspectives on Agent Evolution and its Safety

From tabular reinforcement learning to constrained sequential evolution

Paper Sharing

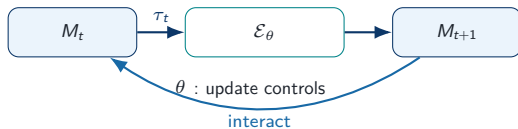
Theory-first reading of Sutton & Barto and related work

Xianzhe Meng

October 2026

Chapter 1 | The question behind the whole talk

What exactly is being optimized, estimated, approximated, or constrained?



reward, gain, value, quality

$$R_t = r(s_t, a_t) \quad G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Key idea

For a self-evolving agent, the update rule is part of the agent. Safety therefore concerns the dynamics of optimization, not only the final policy.

Evolution as a parameterized operator

$$M_{t+1} = \mathcal{E}(M_t, \tau_t; \theta_t), \quad \tau_t = (S_0, A_0, R_1, \dots, S_t).$$

- ▶ M_t can contain policy parameters, memory, skills, a world model, or a rule for updating them.
- ▶ θ_t controls how strongly the agent changes after evidence.
- ▶ The data distribution changes with M_t because the agent changes what it visits.

Key idea

An evolution operator is a dynamical system driven by an optimization signal.

Same data, different objectives

Utility-only update

$$\theta_{t+1} = \theta_t + \alpha \nabla \widehat{J_R}(\theta_t)$$

Cost-penalized update

$$\theta_{t+1} = \theta_t + \alpha \nabla \widehat{J_R}(\theta_t) - \eta \nabla \widehat{J_C}(\theta_t)$$

objective $\Rightarrow$ trajectory distribution $\Rightarrow$ safety outcome

Safety question

Changing the objective changes the path. A cost penalty alone does not enforce a hard safety budget.

Reward hacking and tampering

- ▶ The optimizer follows the written objective, including its blind spots.
- ▶ A proxy objective may be easier to improve than the intended property.
- ▶ In a sequential system, the agent can change the state distribution that defines the proxy.

intended utility $U \neq$ implemented reward $R \implies \arg \max_{\pi} J_R(\pi)$ need not maximize $J_U(\pi)$.

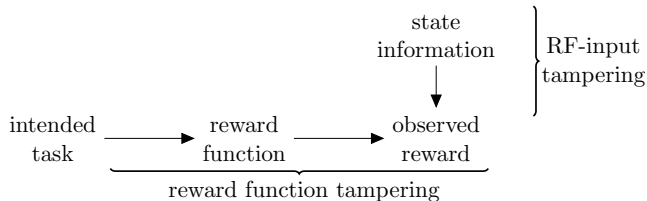
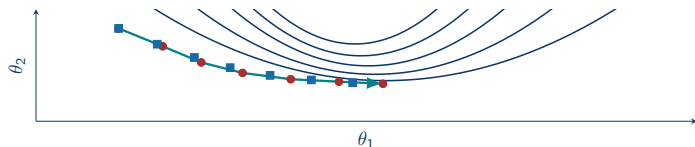


Figure 1.: Reward tampering subproblems. Loosely, *reward function tampering* means inappropriately influencing the implemented reward function, while *RF-input tampering* means inappropriately influencing the information that the reward function has about the environment state.

Safety depends on the evolution path

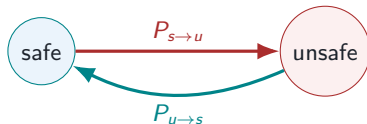


- ▶ The same objective can produce different evolution paths under different update rules.
- ▶ A path can cross unsafe regions even if its endpoint is feasible.

Safety has a temporal direction

$P_{s \rightarrow u} = P(\text{safe before update} \rightarrow \text{unsafe after update}),$

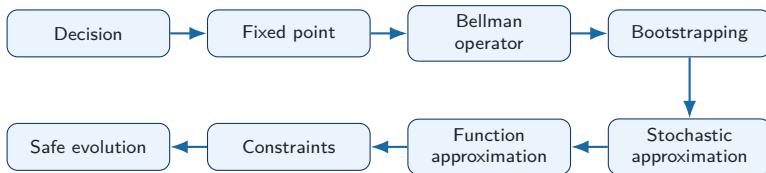
$P_{u \rightarrow s} = P(\text{unsafe before update} \rightarrow \text{safe after update}).$



Key idea

A scalar penalty can hide this asymmetry. A transition-sensitive safety objective exposes it.

The common spine



Key idea

The chapters are different numerical views of one sequential optimization problem.

Source / pointer: Sutton and Barto, Chapters 3–12; Bertsekas and Tsitsiklis.

Five objects that must not be confused

objective	$J(\theta)$	which solution is preferred?
fixed-point equation	$v = Tv$	which value should be represented?
sample target	$Y_t \approx (Tv)(S_t)$	what information is observed?
numerical update	$\theta_{t+1} = \theta_t + \alpha_t \Delta_t$	how does the estimate move?
safety condition	$C(\theta) \leq d$	which solutions or updates are admissible?

Key idea

An algorithm can estimate the right fixed point with a noisy target, use a stable update, and still violate the safety condition.

The causal chain used in every chapter

decision problem $\implies$ equation to solve $\implies$ available information

available information $\implies$ update rule $\implies$ behavior and safety

- optimization asks: which solution is desirable?
- numerical analysis asks: does the update reach that solution?
- safety asks: are both the path and the solution admissible?

Four questions for every algorithm

1. **What does it solve?**

Which objective and which equation define the desired solution?

2. **What does it estimate?**

Which sample target is available, and which quantity does it estimate?

3. **How does it update?**

Which variables change, with what step size and information reuse?

4. **Where is safety enforced?**

Which policies, actions, or agent updates are admissible?

Reading rule

Keep objective, fixed point, sample target, update rule, and safety constraint separate.

Why classical RL is a natural starting point

- ▶ Tabular RL isolates the update geometry from representation learning.
- ▶ Bellman equations provide exact fixed points and contraction arguments.
- ▶ Stochastic approximation exposes the role of step size, noise, and visitation.

tabular RL → optimization primitives → safe evolution

Consequence: CMDPs later make safety a feasible-set problem, not merely a smaller reward.

What Is the 'Table' in Tabular RL?

1 Core Idea State set: $\mathcal{S} = \{s_1, s_2, s_3, s_4\}$ Action set: $\mathcal{A} = \{a_1, a_2, a_3\}$

$V(s)$: one value per state

State s	$V(s)$
s_1	0.8
s_2	0.1
s_3	0.5
s_4	-0.2

$Q(s, a)$: one value per state-action pair

State s	a_1	a_2	a_3
s_1	0.7	0.2	-0.1
s_2	0.0	0.9	0.4
s_3	0.3	-0.2	0.1
s_4	-0.5	0.6	0.2

The table is a lookup map from discrete states (or state-action pairs) to stored estimates.

2 How the table is updated

DP	TD	MC
Full expectation from a model	One real sample + bootstrap	Wait for full return
$r + \gamma \sum_{s'} P(s' s, a) V(s')$	$r + \gamma V(s')$	G_t

All of them update entries in the same table.

3 Geometric intuition Update one cell at a time

	a_1	a_2	a_3
s_1	...	...	...
s_2	...	0.9	...
s_3	...	...	...
s_4	...	...	...

Tabular methods do not learn features. They directly adjust stored numbers.

Works when the state/action space is small and discrete.

4 Final takeaway

One sentence: In tabular RL, learning means repeatedly updating the numbers inside a lookup table.

Safety as constrained sequential optimization

$$\max_{\theta} J_R(\theta) \quad \text{s.t.} \quad J_C(\theta) \leq d.$$

$$\mathcal{L}(\theta, \lambda) = J_R(\theta) - \lambda(J_C(\theta) - d),$$

Reward ranks candidate policies or updates; the cost budget defines admissibility.

The multiplier expresses the price of violating the budget.

Key idea

Safe evolution must account for both useful improvement and admissible update dynamics.

KKT conditions and primal-dual updates are developed in Chapter 10.

Chapter 1 takeaway

Safe evolution is a constrained sequential optimization problem.

Decision making $\rightarrow$ fixed points $\rightarrow$ stochastic updates $\rightarrow$ constraints $\rightarrow$ safe evolution

Key idea

For each algorithm: what does it solve, what does it estimate, how does it update, and where is safety enforced?

Why the next chapter follows. Bandits isolate uncertainty in action values before state dynamics and long-horizon credit assignment are introduced.

How should trials be allocated when action values are unknown?

known means: $a^* \in \arg \max_a q_*(a),$

unknown means: $q_*(a) = \mathbb{E}[R_t | A_t = a]$ must be estimated from chosen actions,

consequence: each action trades immediate reward for information.

Key idea

Exploration appears because the decision changes the data used by the estimator.

Sample-average estimation

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{n}(R_n - Q_n(a)).$$

- ▶ The update is stochastic approximation to the root $\mathbb{E}[R - Q] = 0$.
- ▶ The step size shrinks because the objective is a stationary mean.
- ▶ Greedy choice optimizes the current estimate but stops collecting information.

Constant step size and tracking

$$Q_{n+1} = Q_n + \alpha(R_n - Q_n), \quad 0 < \alpha \leq 1.$$

- ▶ Constant α never fully forgets recent observations.
- ▶ It tracks drifting rewards but retains a stationary noise floor.
- ▶ In evolution, this is a simple model of a memory update with finite plasticity.

Optimization interpretation

Q minimizes a streaming squared error with an exponentially weighted history.

The stochastic-approximation error recursion

$$Q_{n+1} - q_* = (1 - \alpha)(Q_n - q_*) + \underbrace{\alpha(R_n - q_*)}_{\xi_n}.$$

$$\mathbb{E}[Q_n - q_*] = (1 - \alpha)^n(Q_0 - q_*),$$

$$\begin{aligned}\text{Var}(Q_n) &= \alpha^2 \sigma^2 \sum_{i=0}^{n-1} (1 - \alpha)^{2i} \\ &\rightarrow \frac{\alpha}{2 - \alpha} \sigma^2.\end{aligned}$$

Key idea

Constant α creates a bias-memory tradeoff: it forgets quickly, but it never removes stationary noise.

Robbins–Monro step-size conditions

$$Q_{n+1} = Q_n + \alpha_n(h(Q_n) + \xi_n), \quad h(q_*) = 0.$$

$$\sum_{n=1}^{\infty} \alpha_n = \infty, \quad \sum_{n=1}^{\infty} \alpha_n^2 < \infty.$$

- ▶ The first sum prevents premature freezing.
- ▶ The second sum makes accumulated martingale noise finite.

Source / pointer: Robbins and Monro (1951); Sutton and Barto, Eq. 2.7.

ε -greedy: exploration as randomized optimization

$$\pi(a \mid Q) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}|, & a \in \arg \max_b Q(b), \\ \varepsilon/|\mathcal{A}|, & \text{otherwise.} \end{cases}$$

- ▶ Exploitation follows the current optimizer.
- ▶ Exploration perturbs the decision rule, not the objective estimate.
- ▶ With safety costs, random perturbations can leave the admissible action set.

Optimistic initialization

$$Q_0(a) = Q_{\text{high}} \quad \text{for every action.}$$

- ▶ Untried actions look attractive, so greedy behavior creates its own exploration.
- ▶ The optimism is an implicit prior, not a confidence interval with a calibrated probability.
- ▶ If high estimates encourage unsafe trials, optimism becomes unsafe exploration.

Source figure: optimistic initial values

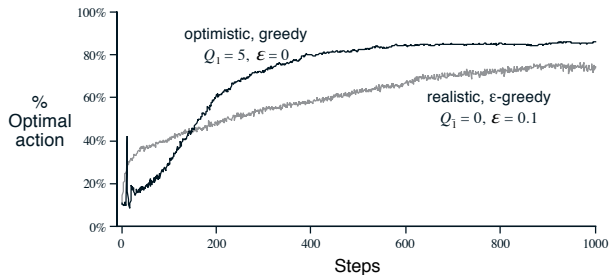


Figure 2.2: The effect of optimistic initial action-value estimates on the 10-armed testbed. Both methods used a constant step-size parameter, $\alpha = 0.1$.

Source: Sutton and Barto, Reinforcement Learning: An Introduction, 2nd ed., Figure 2.2, p. 40 of the downloaded draft. The figure compares optimistic greedy and realistic ϵ -greedy on the 10-armed testbed.

UCB as an uncertainty-regularized optimizer

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right].$$

exploitation

$Q_t(a)$

estimated value

exploration bonus

$c \sqrt{\ln t / N_t(a)}$

uncertainty proxy

Key idea

UCB replaces blind randomness with a deterministic optimism rule. The safety question becomes: what confidence set is admissible?

UCB from a confidence event

$R_i(a) \in [0, 1]$ independently, conditional on choosing a .

(Hoeffding inequality) $\Pr(|Q_t(a) - q_*(a)| \geq \varepsilon) \leq 2 \exp(-2N_t(a)\varepsilon^2)$

$$2 \exp(-2N_t(a)\varepsilon^2) = \frac{2}{t}$$

$$\varepsilon_t(a) = \sqrt{\frac{\ln t}{2N_t(a)}} \implies q_*(a) \leq Q_t(a) + \varepsilon_t(a)$$

$$U_t(a) = Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}}.$$

Key idea

The bonus is a high-probability upper bound, not arbitrary exploration noise.

Gradient bandit: the score-function derivation

$$\begin{aligned} J(H) &= \sum_a \pi_H(a) q_*(a), \\ \frac{\partial J}{\partial H(b)} &= \sum_a q_*(a) \frac{\partial \pi_H(a)}{\partial H(b)} \qquad \pi_H(a) = \frac{e^{H(a)}}{\sum_c e^{H(c)}} \\ &= \mathbb{E}_{A \sim \pi_H} [q_*(A) (\mathbf{1}\{A = b\} - \pi_H(b))] . \\ q_*(A) &\rightsquigarrow R_t - \bar{R}_t \end{aligned}$$

Key idea

The gradient bandit update is a stochastic gradient estimator in preference space.

Bandits and safe exploration

$$\max_a \mathbb{E}[R(a)] \quad \text{s.t.} \quad \mathbb{P}(C(a) > d) \leq \delta.$$

- ▶ Uncertainty bonuses should be built around both reward and cost.
- ▶ A high reward estimate is not enough if its confidence interval overlaps catastrophic cost.
- ▶ Bandits foreshadow CMDPs: optimize a primary quantity inside a safe information set.

Chapter 2 takeaway

bandit learning = estimation + decision under uncertainty

- ▶ Greedy, UCB, and gradient bandits differ in how they convert uncertainty into updates.
- ▶ Safe evolution must control not only the expected objective but also the risk of information-gathering actions.

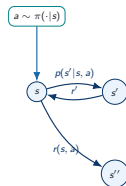
Why the next chapter follows. Bandits cannot express how an action changes future states. MDPs add that missing causal link.

Chapter 3 | MDP

bandit: $A_t \longrightarrow R_{t+1},$

MDP: $(S_t, A_t) \longrightarrow S_{t+1} \longrightarrow R_{t+2}, R_{t+3}, \dots,$

therefore: $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s].$



decision $\rightarrow$ transition $\rightarrow$ reward $\rightarrow$ next decision

Key idea

The value appears on both sides because future decisions depend on the next state.

Value functions

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s], \quad q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

$$v_{\pi}(s) = \sum_a \pi(a \mid s) q_{\pi}(s, a).$$

- ▶ v_{π} evaluates states under a policy.
- ▶ q_{π} evaluates one-step deviations before policy continuation.

Bellman expectation equation

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]. \end{aligned}$$

Talk: The first line is the algebraic hinge: split the return at one step. The second line applies the Markov property.

Bellman operators

$$(T^\pi V)(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')],$$

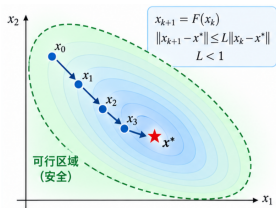
$$(TV)(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

$$v_\pi = T^\pi v_\pi, \quad v_* = Tv_*.$$

Key idea

Policy evaluation and control are fixed-point problems for different operators.

Fixed points, residuals, and iteration



$$V^*(s) = \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma V^*(s')]$$
$$Q^*(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma \max_{a'} Q^*(s', a')]$$

$$F(x_*) = x_*, \quad r(x) = F(x) - x, \quad x_{k+1} = F(x_k).$$

$$\begin{aligned} \|x_{k+1} - x_*\| &= \|F(x_k) - F(x_*)\| \\ &\leq L \|x_k - x_*\|, \quad L < 1. \end{aligned}$$

$$\|x_k - x_*\| \leq L^k \|x_0 - x_*\|.$$

Key idea

The optimization view starts by treating an update as an operator with a fixed point, a residual, and a stability constant.

Contraction proof in three lines

Assume F maps a complete space into itself and has contraction factor $0 \leq L < 1$.

$$\|F(x) - F(y)\| \leq L\|x - y\|,$$

$$\|x_{k+1} - x_k\| \leq L^k \|x_1 - x_0\|,$$

$$\|x_m - x_n\| \leq \sum_{k=n}^{m-1} L^k \|x_1 - x_0\|.$$

$$\Rightarrow \{x_k\} \text{ is Cauchy} \Rightarrow x_k \rightarrow x_*.$$

RL reading

Discounting supplies the contraction factor $L = \gamma$ for Bellman evaluation.

Contraction: the key inequality

$$\begin{aligned} |(T^\pi V)(s) - (T^\pi U)(s)| &\leq \sum_{s'} \Pr(s' \mid s, \pi) \gamma |V(s') - U(s')| \\ &\leq \gamma \|V - U\|_\infty. \end{aligned}$$

$$\|T^\pi V - T^\pi U\|_\infty \leq \gamma \|V - U\|_\infty.$$

Banach Fixed-Point Theorem

Preliminaries:

Let (X, d) be a non-empty **complete metric space**.

Let $T : X \rightarrow X$ be a mapping. T is called a **contraction mapping** if there exists a constant $\gamma \in [0, 1)$ such that for all $x, y \in X$:

$$d(Tx, Ty) \leq \gamma d(x, y)$$

Theorem Statement:

If T is a contraction mapping on a complete metric space (X, d) , then:

1. **Existence and Uniqueness:** There exists a unique fixed point $x^* \in X$ such that $T(x^*) = x^*$.
2. **Convergence:** For any arbitrary initial point $x_0 \in X$, the sequence defined by $x_{n+1} = T(x_n)$ converges to the unique fixed point x^* .
3. **Rate of Convergence:** The convergence is geometric (exponential). Specifically,
$$d(x_n, x^*) \leq \gamma^n d(x_0, x^*)$$

and
$$d(x_n, x^*) \leq \frac{\gamma^n}{1-\gamma} d(x_0, x_1)$$

Why max preserves contraction

$$\left| \max_a x_a - \max_a y_a \right| \leq \max_a |x_a - y_a|.$$

$$\begin{aligned} |(TV)(s) - (TU)(s)| &\leq \max_a \gamma \sum_{s'} p(s' | s, a) |V(s') - U(s')| \\ &\leq \gamma \|V - U\|_{\infty}. \end{aligned}$$

Key idea

The Bellman optimality operator is nonlinear because of the max, but still contractive under the sup norm.

Optimality equations

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')],$$
$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')].$$

- ▶ v_* is a fixed point in state space.
- ▶ q_* folds action selection into the recursion.
- ▶ The operator itself encodes a one-step lookahead optimizer.

Bellman residual as an optimization diagnostic

$$\mathcal{R}_\pi(V) = T^\pi V - V, \quad \mathcal{R}_*(V) = TV - V.$$

- ▶ A zero residual means a fixed point, not necessarily a good representation under function approximation.
- ▶ Residual norms can be small while policy performance is sensitive to distribution shift.
- ▶ Safety variants can define residuals for reward and cost separately.

Chapter 3 takeaway

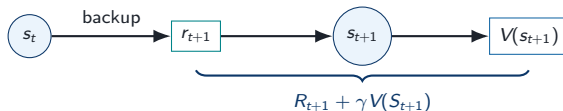
sequential decision making $\longrightarrow$ Bellman fixed point

- ▶ DP applies the operator exactly.
- ▶ MC estimates the full return.
- ▶ TD samples a one-step operator application.

Key idea

The next chapters compare numerical methods for solving or approximating the same underlying fixed-point equations.

equation from Chapter 3: $v_{\pi} = T^{\pi} v_{\pi},$
extra information now available: $p(s', r | s, a),$
solver: $V_{k+1} = T^{\pi} V_k.$



Key idea

Dynamic programming converts a Bellman equation into a deterministic fixed-point iteration.

Policy evaluation as linear fixed-point iteration

$$V_{k+1} = T^\pi V_k = r^\pi + \gamma P^\pi V_k.$$
$$(I - \gamma P^\pi) V_\pi = r^\pi.$$

- ▶ Direct solve: factor a linear system.
- ▶ Iteration: repeatedly apply a contraction.
- ▶ The choice resembles Jacobi, Gauss–Seidel, or asynchronous relaxation.

Numerical bridge: old versus fresh values

Fixed-point iteration

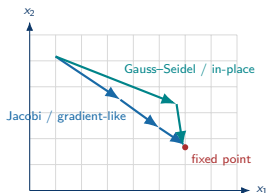
$$x_{k+1} = F(x_k)$$

$$V_{k+1} = T^\pi V_k$$

Root finding / optimization

$$F(x_*) = x_*$$

$$\nabla J(\theta_*) = 0$$



Key idea

Jacobi-like updates use old coordinates; Gauss-Seidel-like updates use fresh coordinates. RL algorithms differ in exactly this kind of information timing.

Jacobi-style policy evaluation

$$V_{k+1}(s) = r^\pi(s) + \gamma \sum_{s'} P^\pi(s' | s) V_k(s').$$

- ▶ Every state reads the previous sweep V_k .
- ▶ The update is easy to parallelize.
- ▶ Contraction gives a clean sup-norm bound.

$$\|V_k - V_\pi\|_\infty \leq \gamma^k \|V_0 - V_\pi\|_\infty.$$

$$\begin{cases} 10x_1 - x_2 + x_3 = 11 \\ -x_1 + 10x_2 - x_3 = 16 \\ x_1 - x_2 + 10x_3 = 29 \end{cases}$$

$$\begin{cases} x_1^{(k+1)} = 1.1 + 0.1x_2^{(k)} - 0.1x_3^{(k)} \\ x_2^{(k+1)} = 1.6 + 0.1x_1^{(k)} + 0.1x_3^{(k)} \\ x_3^{(k+1)} = 2.9 - 0.1x_1^{(k)} + 0.1x_2^{(k)} \end{cases}$$

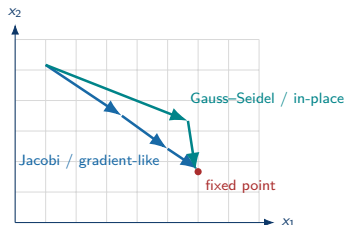
Gauss–Seidel-style evaluation

$$V_{k+1}(s_i) = r^\pi(s_i) + \gamma \left(\sum_{j < i} P_{ij}^\pi V_{k+1}(s_j) + \sum_{j \geq i} P_{ij}^\pi V_k(s_j) \right).$$

- ▶ The sweep uses fresh values as soon as they exist.
- ▶ In-place TD-like methods can be viewed as asynchronous local relaxations.
- ▶ The ordering can change practical speed even when the fixed point is unchanged.

$$\begin{cases} 10x_1 - x_2 + x_3 = 11 \\ -x_1 + 10x_2 - x_3 = 16 \\ x_1 - x_2 + 10x_3 = 29 \end{cases}$$

$$\begin{cases} x_1^{(k+1)} = 1.1 + 0.1x_2^{(k)} - 0.1x_3^{(k)} \\ x_2^{(k+1)} = 1.6 + 0.1x_1^{(k+1)} + 0.1x_3^{(k)} \\ x_3^{(k+1)} = 2.9 - 0.1x_1^{(k+1)} + 0.1x_2^{(k+1)} \end{cases}$$



Gauss-Seidel: the linear-system view

$$\begin{cases} 10x_1 - x_2 + x_3 = 11 \\ -x_1 + 10x_2 - x_3 = 16 \\ x_1 - x_2 + 10x_3 = 29 \end{cases}$$

$$A = D + L + U, \quad Ax = b.$$

$$\begin{cases} x_1^{(k+1)} = 1.1 + 0.1x_2^{(k)} - 0.1x_3^{(k)} \\ x_2^{(k+1)} = 1.6 + 0.1x_1^{(k+1)} + 0.1x_3^{(k)} \\ x_3^{(k+1)} = 2.9 - 0.1x_1^{(k+1)} + 0.1x_2^{(k+1)} \end{cases}$$

$$x^{k+1} = -(D + L)^{-1} U x^k + (D + L)^{-1} b,$$

$$B_{GS} = -(D + L)^{-1} U.$$

$$\rho(B_{GS}) < 1 \quad \Rightarrow \quad x^k \rightarrow A^{-1}b.$$

RL reading

An asynchronous backup reuses newly updated coordinates. The update order changes the iteration matrix and therefore the transient behavior.

Policy improvement theorem

$$T^{\pi'} V = r^{\pi'} + \gamma P^{\pi'} V$$

$$v_{\pi} \leq T^{\pi'} v_{\pi}$$

$$T^{\pi'} v_{\pi} \leq (T^{\pi'})^2 v_{\pi}$$

$$v_{\pi} \leq T^{\pi'} v_{\pi} \leq (T^{\pi'})^2 v_{\pi} \leq \dots \leq (T^{\pi'})^k v_{\pi}$$

$$\lim_{k \rightarrow \infty} (T^{\pi'})^k v_{\pi} = v_{\pi'}$$

$$v_{\pi} \leq v_{\pi'}$$

$$\pi'(s) \in \arg \max_a q_{\pi}(s, a).$$

$$v_{\pi}(s) \leq q_{\pi}(s, \pi'(s))$$

$$= r(s, \pi'(s)) + \gamma \sum_{s'} p(s' \mid s, \pi'(s)) v_{\pi}(s')$$

$$\leq v_{\pi'}(s).$$

Talk: The first inequality is one-step greedification. Repeating it yields monotone improvement under the exact model.

Policy iteration

1. Evaluate V^{π_k} until the policy fixed point is solved.
2. Improve with $\pi_{k+1}(s) \in \arg \max_a q_{\pi_k}(s, a)$.
3. Stop when $\pi_{k+1} = \pi_k$.

$$\pi_k \rightarrow V^{\pi_k} \rightarrow \pi_{k+1} \rightarrow \dots$$

Key idea

Policy iteration alternates between solving an evaluation subproblem and a greedy optimization subproblem.

Value iteration

$$V_{k+1} = TV_k, \quad V_{k+1}(s) = \max_a \{ r(s, a) + \gamma \sum_{s'} p(s' | s, a) V_k(s') \}.$$

- ▶ Each sweep performs one optimality backup.
- ▶ It avoids exact policy evaluation.
- ▶ It is a nonlinear contraction iteration because of the max.

Generalized policy iteration

$$\underbrace{\text{partial evaluation}}_{V \leftarrow T^\pi V} + \underbrace{\text{partial improvement}}_{\pi \leftarrow \text{greedy}(V)}.$$

- ▶ Policy iteration and value iteration are endpoints of a continuum.
- ▶ SARSA, Q-learning, actor–critic, and trace methods instantiate approximate GPI.
- ▶ Safety constraints add a second value or a feasibility mechanism to the loop.

$$V_{k+1}^R = T_R V_k^R, \quad V_{k+1}^C = T_C V_k^C.$$

- ▶ A safe backup must propagate cost-to-go, not only reward-to-go.
- ▶ Feasibility can be checked after a full backup or enforced during the backup.
- ▶ The latter resembles a constrained Bellman operator and can be more conservative.

Chapter 4 takeaway

DP = exact Bellman operator iteration

- ▶ Contraction supplies stability.
- ▶ Update ordering supplies speed and practical behavior.
- ▶ Gauss–Seidel intuition helps explain why in-place, asynchronous, and bootstrapped methods can behave differently.

Why the next chapter follows. Exact Bellman backups require a known model. Monte Carlo replaces the model expectation with sampled returns.

DP requires: $T^\pi V = \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t],$

model unavailable: the conditional expectation cannot be computed,

sample replacement: $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}, \quad \mathbb{E}[G_t \mid S_t = s] = v_\pi(s).$

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)].$$

Key idea

Monte Carlo removes model bias by accepting trajectory variance and delayed updates.

Return decomposition

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T,$$
$$\mathbb{E}_\pi[G_t \mid S_t = s] = v_\pi(s).$$

$$\hat{v}_n(s) = \frac{\sum_{i=1}^n G_t^{(i)} \mathbf{1}\{S_t^{(i)} = s\}}{\sum_{i=1}^n \mathbf{1}\{S_t^{(i)} = s\}}.$$

- ▶ No model is required.
- ▶ No bootstrap target appears.
- ▶ The episode must terminate, or a truncation convention must define G_t .

Unbiasedness of the MC target

$$\begin{aligned}\mathbb{E}[G_t \mid S_t = s] &= \mathbb{E}\left[\sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \mid S_t = s\right] \\ &= v_\pi(s).\end{aligned}$$

$$\text{Bias}[G_t \mid S_t = s] = 0, \quad \text{Var}[G_t \mid S_t = s] \text{ can be large.}$$

Key idea

MC removes one-step model bias by accepting long-horizon sampling variance.

MC control as policy optimization

$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha_k [G_t - Q_k(s, a)], \quad \pi_{k+1} = \text{Greedy}(Q_k).$$

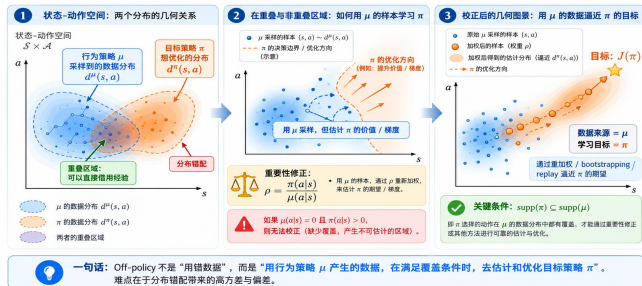
$$Q^{\pi_k} \xrightarrow{\text{MC evaluation}} \hat{Q}_k \xrightarrow{\text{greedy improvement}} \pi_{k+1}.$$

GPI interpretation

Evaluation is noisy and incomplete; improvement is exact only with infinite data.

Off-policy MC and importance sampling

Off-policy 的几何直观



$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}.$$

$$\mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_b[\rho_{t:T-1} G_t | S_t = s].$$

- ▶ The ratio changes the trajectory measure from behavior b to target π .
- ▶ Support condition: $\pi(a | s) > 0 \Rightarrow b(a | s) > 0$.

Why importance sampling can explode

$$\text{Var}_b[\rho G] = \mathbb{E}_b[\rho^2 G^2] - (\mathbb{E}_b[\rho G])^2.$$

$$\mathbb{E}_b[\rho^2] = \mathbb{E}_b \left[\prod_{k=t}^{T-1} \left(\frac{\pi(A_k | S_k)}{b(A_k | S_k)} \right)^2 \right].$$

Key idea

Long horizons multiply density ratios. A mathematically unbiased correction can be statistically unusable.

MC versus DP

DP target

$$R_{t+1} + \gamma V(S_{t+1})$$

low sampling variance, model required

MC target

$$G_t$$

unbiased return, high variance

MC chooses target fidelity over target locality.

$$\hat{J}_C(\pi) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \gamma^t C_t^{(i)}.$$

- ▶ Full cost returns can reveal rare catastrophic trajectories.
- ▶ With small event probability, the confidence interval can remain wide for a long time.
- ▶ A safe update needs a confidence margin, not just an empirical mean.

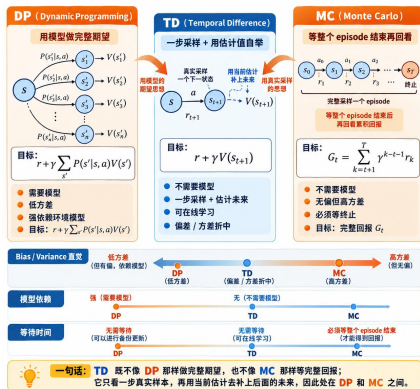
Chapter 5 takeaway

MC = unbiased return estimation + long-horizon variance

- ▶ MC is an estimator of a fixed point, not a Bellman solver with a model.
- ▶ Importance sampling changes the measure but may destroy variance control.
- ▶ TD will trade this variance for bootstrap bias.

Chapter 6 | TD

TD: DP 和 MC 之间的 trade-off



MC target: G_t available at episode end,

Bellman identity: $v_{\pi}(S_t) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t],$

TD target: $Y_t = R_{t+1} + \gamma V_t(S_{t+1})$ available after one step.

$$\delta_t = Y_t - V_t(S_t), \quad V_{t+1}(S_t) = V_t(S_t) + \alpha_t \delta_t.$$

Key idea

TD gains online learning by replacing an unknown future value with the current estimate.

One fixed point, three computational routes

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s].$$

DP:	$\sum_{s', r} p(s', r \mid s, \pi)[r + \gamma V(s')]$	exact expectation,
MC:	G_t	sample the complete return,
TD:	$R_{t+1} + \gamma V(S_{t+1})$	sample one step and bootstrap.

Key idea

DP, MC, and TD differ in target construction, not in the value function they seek in the tabular on-policy case.

TD as a stochastic Bellman residual step

$$\begin{aligned}\delta_t &= R_{t+1} + \gamma V(S_{t+1}) - V(S_t), \\ \mathbb{E}[\delta_t \mid S_t = s] &= (T^\pi V)(s) - V(s). \\ V_{t+1}(s) &= V_t(s) + \alpha_t((T^\pi V_t)(s) - V_t(s) + \xi_t).\end{aligned}$$

Key idea

The mean TD update points toward the Bellman fixed point; the sample update adds martingale noise.

TD(0) and the projected fixed point

$$V_{t+1} = V_t + \alpha_t e_{S_t} (R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)).$$

$$\Delta V_t = V_{t+1} - V_t = \alpha_t e_{S_t} \delta_t$$

$$\mathbb{E}[\Delta V_t | V_t] = \sum_s \underline{\mu(s)} \alpha_t e_s \mathbb{E}[\delta_t | S_t = s, V_t]$$

$$\mathbb{E}[\Delta V_t | V_t] \approx \alpha_t D_\mu (T^\pi V_t - V_t),$$

distribution of state

$$D_\mu = \text{diag}(\mu(s)).$$

$$\mathbb{E}[\Delta V_t | V_t] = \alpha_t D_\mu (T^\pi V_t - V_t)$$

$V \rightarrow V_\pi$ under tabular coverage and Robbins–Monro conditions.

The TD error is a temporal derivative

$$\delta_t = \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{one-step target}} - \underbrace{V(S_t)}_{\text{current prediction}}.$$

$$G_t - V(S_t) = \delta_t + \gamma(G_{t+1} - V(S_{t+1})),$$

$$G_t - V(S_t) = \sum_{k=0}^{T-t-1} \gamma^k \delta_{t+k}.$$

Key idea

The full-return error telescopes into discounted TD errors.

TD versus MC: algebraic identity

$$G_t = V(S_t) + \sum_{k=0}^{T-t-1} \gamma^k \delta_{t+k}.$$

- ▶ MC uses the entire sum as one target.
- ▶ TD(0) uses only the first term δ_t .
- ▶ n -step TD and traces choose how much of the sum to retain.

TD error sequence $\Longleftrightarrow$ return error decomposition.

Why TD can learn online

after observing (S_t, R_{t+1}, S_{t+1}) : $V(S_t) \leftarrow V(S_t) + \alpha \delta_t$.

- ▶ The update does not wait for S_T .
- ▶ The next-state estimate supplies information about the unfinished tail.
- ▶ The price is bias from using an imperfect V inside the target.

TD(0) convergence proof skeleton

$$V_{t+1} - V_\pi = (V_t - V_\pi) + \alpha_t [D_\mu(T^\pi V_t - V_t) + \xi_t].$$

$$\langle V - V_\pi, T^\pi V - V \rangle_{D_\mu} \leq -(1 - \gamma) \|V - V_\pi\|_{D_\mu}^2 < 0 \quad \text{for } V \neq V_\pi,$$

$$\text{because } \mu^\top P^\pi = \mu^\top \text{ and } \|P^\pi e\|_{D_\mu} \leq \|e\|_{D_\mu}.$$

- ▶ drift points toward the fixed point;
- ▶ noise has bounded conditional second moments;
- ▶ step sizes satisfy $\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$.

SARSA: on-policy action-value iteration

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

$$\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = (T^\pi Q)(s, a) - Q(s, a).$$

$$Q^\pi = T^\pi Q^\pi.$$

Q-learning: off-policy optimality iteration

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)].$$

$$\mathbb{E}[\delta_t \mid S_t = s, A_t = a] = (TQ)(s, a) - Q(s, a).$$

Key idea

Behavior may explore with ϵ -greedy, while the target evaluates the greedy policy.

Maximization bias

$$\mathbb{E} \left[\max_a \hat{Q}(a) \right] \geq \max_a \mathbb{E}[\hat{Q}(a)].$$

$$\hat{Q}(a) = Q(a) + \varepsilon_a, \quad \mathbb{E}[\varepsilon_a] = 0,$$

max target selects positive noise more often than negative noise.

Example:

$$\hat{Q}(a_1) = 0.8, \quad \hat{Q}(a_2) = -0.5$$

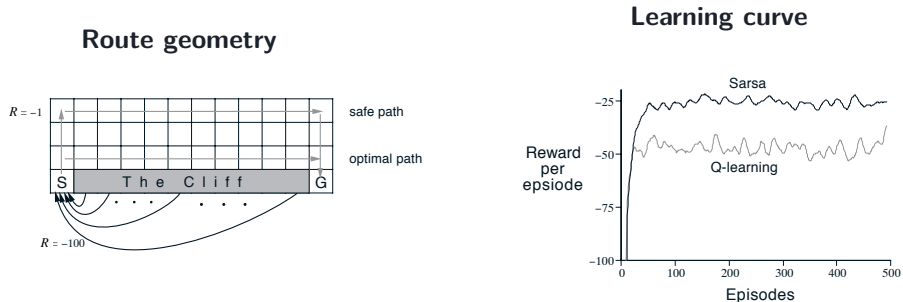
$$\hat{Q}(a_1) = -0.2, \quad \hat{Q}(a_2) = 0.3$$

$$\hat{Q}(a_1) = 0.1, \quad \hat{Q}(a_2) = -0.1$$

Key idea

The nonlinear optimizer and the estimator interact: maximization converts zero-mean noise into positive bias.

Original source figure: cliff walking



on-policy behavior $\implies$ safer transient path.

Source: Sutton and Barto, Figure 6.13, Cliff Walking. The original figure compares Sarsa and Q-learning, making on-policy risk sensitivity visible.

Policy dependence of the safety outcome

$$J_R(\pi) = \mathbb{E}_\pi \left[\sum_t \gamma^t R_{t+1} \right], \quad J_C(\pi) = \mathbb{E}_\pi \left[\sum_t \gamma^t C_{t+1} \right].$$

$\pi_{\text{safe}} = \arg \max_{\pi} J_R(\pi)$ within a conservative behavior class,

$\pi_{\text{greedy}} = \arg \max_{\pi} J_R(\pi)$ under the target operator.

Key idea

SARSA and Q-learning can solve the same MDP but optimize different policy semantics during learning.

Chapter 6 takeaway

$$\text{TD} = \text{sampld Bellman residual} + \text{bootstrap}$$

- ▶ TD moves online and locally.
- ▶ SARSA, Expected SARSA, and Q-learning differ in target construction.
- ▶ Safety depends on whether the update evaluates the behavior policy, a greedy policy, or a constrained policy.

Why the next chapter follows. One-step TD commits to a single bootstrap horizon. The next chapter makes that horizon adjustable.

Chapter 7 | N step-TD

TD(0): $R_{t+1} + \gamma V(S_{t+1})$ short delay, more bootstrap error,

MC: G_t long delay, more sampling variance,

interpolation: $G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}).$

Key idea

Changing n changes the estimator and its error profile, while the Bellman fixed point stays the same.

The n-step return

$$G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1}),$$

$$G_{t:t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}),$$

$$G_{t:t+n} = R_{t+1} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

$$V(S_t) \leftarrow V(S_t) + \alpha [G_{t:t+n} - V(S_t)].$$

Telescoping relation

$$G_{t:t+n} - V(S_t) = \delta_t + \gamma\delta_{t+1} + \cdots + \gamma^{n-1}\delta_{t+n-1}.$$

$$G_{t:t+n} = V(S_t) + \sum_{k=0}^{n-1} \gamma^k \delta_{t+k}.$$

Key idea

The n-step target is a finite window over the TD-error sequence.

A continuum of targets



Bias of a bootstrap target

$$\mathbb{E}[G_{t:t+n} \mid S_t = s] - v_\pi(s) = \gamma^n \mathbb{E}[V(S_{t+n}) - v_\pi(S_{t+n}) \mid S_t = s].$$

$$|\text{Bias}_n(s)| \leq \gamma^n \|V - v_\pi\|_\infty.$$

Key idea

Longer backups attenuate the current value error by γ^n .

Variance of the n-step target

$$G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}).$$

$$\text{Var}(G_{t:t+n}) \leq \left(\sum_{k=0}^{n-1} \gamma^k \sqrt{\text{Var}(R_{t+k+1})} + \gamma^n \sqrt{\text{Var}(V(S_{t+n}))} \right)^2.$$

This is Minkowski's inequality in L^2 ; no independence assumption is needed.

$n \uparrow \Rightarrow$ more sampled rewards enter the target.

The numerical view

$\text{TD}(0) \leftrightarrow n\text{-step operator splitting} \leftrightarrow \text{MC sample solve.}$

- ▶ n controls how many future coordinates enter the update.
- ▶ The update is still a stochastic approximation, but with a different mean field.
- ▶ Delayed targets change stability under nonstationary policies.

Chapter 7 takeaway

n -step TD tunes the target, not the underlying Bellman fixed point

- ▶ Small n : online, stable, bootstrap-biased.
- ▶ Large n : return-faithful, delayed, variance-heavy.
- ▶ The next chapter asks how this picture changes when states share parameters.

Chapter 8 | Function Approximation

tabular representation: $V(s) = \theta_s, \quad \dim(\theta) = |\mathcal{S}|,$

scaling obstruction: $|\mathcal{S}|$ is large or continuous,

shared representation: $\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s), \quad \dim(\mathbf{w}) \ll |\mathcal{S}|.$

$$v_\pi = T^\pi v_\pi \quad \longrightarrow \quad \hat{v}_\mathbf{w} = \Pi_\mu T^\pi \hat{v}_\mathbf{w}.$$

Key idea

Parameter sharing introduces projection error and changes the convergence geometry.

Mean-squared value error

$$J(\mathbf{w}) = \frac{1}{2} \mathbb{E}_{S \sim \mu} [(v_{\pi}(S) - \hat{v}(S, \mathbf{w}))^2] .$$

$$\begin{aligned} \nabla J(\mathbf{w}) &= -\mathbb{E}_{\mu} [(v_{\pi}(S) - \hat{v}(S, \mathbf{w})) \mathbf{x}(S)] , \\ \mathbf{w}^+ &= \mathbf{w} + \alpha (v_{\pi}(S) - \hat{v}(S, \mathbf{w})) \mathbf{x}(S) . \end{aligned}$$

Key idea

If the true target were available, this would be ordinary stochastic gradient descent.

Semi-gradient TD

$$U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t), \quad \mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \mathbf{x}(S_t).$$

$$\nabla_{\mathbf{w}} \frac{1}{2} (U_t - \hat{v}(S_t, \mathbf{w}))^2 \neq -[U_t - \hat{v}(S_t, \mathbf{w})] \mathbf{x}(S_t),$$

because U_t also depends on $\mathbf{w}$.

Key idea

The name semi-gradient records exactly which dependence the update ignores.

Projected Bellman equation

$$\hat{v}_{\mathbf{w}} \in \mathcal{V} \xrightarrow{T^\pi} T^\pi \hat{v}_{\mathbf{w}} \notin \mathcal{V} \xrightarrow{\Pi_\mu} \Pi_\mu T^\pi \hat{v}_{\mathbf{w}} \in \mathcal{V}.$$
$$\Pi_\mu T^\pi \hat{v}_{\mathbf{w}} = \hat{v}_{\mathbf{w}}.$$

$$\Pi_\mu f = \arg \min_{g \in \mathcal{V}} \|f - g\|_\mu.$$

$$\hat{v}_{\mathbf{w}_*} = \Pi_\mu T^\pi \hat{v}_{\mathbf{w}_*} \quad \text{generally differs from} \quad \Pi_\mu v_\pi.$$

Key idea

The Bellman step leaves the feature space, so every update implicitly projects back.

Linear TD mean field

$$\delta_t = R_{t+1} + \gamma \mathbf{w}^\top \mathbf{x}_{t+1} - \mathbf{w}^\top \mathbf{x}_t.$$

$$\mathbb{E}[\delta_t \mathbf{x}_t] = \mathbf{b} - A\mathbf{w},$$

$$A = \mathbb{E} [\mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top],$$

$$\mathbf{b} = \mathbb{E}[R_{t+1} \mathbf{x}_t].$$

$$\mathbf{w}_* = A^{-1} \mathbf{b} \quad \text{when } A \text{ is nonsingular.}$$

A stability calculation

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha(\mathbf{b} - A\mathbf{w}_t + \xi_t).$$

$$\mathbb{E}[\mathbf{w}_{t+1} - \mathbf{w}_* \mid \mathbf{w}_t] = (I - \alpha A)(\mathbf{w}_t - \mathbf{w}_*).$$

$\rho(I - \alpha A) < 1$ is the linear mean-stability condition.

- ▶ In tabular on-policy settings, positive definiteness follows from coverage.
- ▶ Off-policy sampling can destroy this geometry.

Local linearization of a nonlinear approximator

$$\hat{v}(s, \mathbf{w} + \Delta) \approx \hat{v}(s, \mathbf{w}) + \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})^T \Delta.$$

$$\Delta = -\alpha \nabla J(\mathbf{w}),$$

$$\hat{v}(s, \mathbf{w} + \Delta) \approx \hat{v}(s, \mathbf{w}) - \alpha \nabla \hat{v}(s, \mathbf{w})^T \nabla J(\mathbf{w}).$$

Key idea

Deep RL updates are locally linear in parameters, even when the global function class is nonlinear.

A local quadratic model

Before constrained policy improvement, approximate the objective near the current parameters.

$$m_{\theta}(\Delta) = J(\theta) + g^{\top} \Delta + \frac{1}{2} \Delta^{\top} H \Delta, \quad g = \nabla J(\theta), \quad H = \nabla^2 J(\theta).$$

$$\nabla_{\Delta} m_{\theta}(\Delta) = g + H \Delta = 0,$$

$$\Delta_N = -H^{-1}g.$$

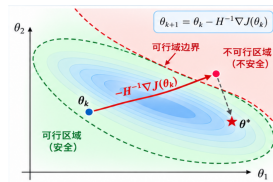
$$\text{gradient step: } \Delta_G = -\alpha g, \quad \text{Newton step: } \Delta_N = -H^{-1}g.$$

Stationarity alone does not imply a maximum. The Hessian must be nonsingular for this Newton step.

Newton iteration: local curvature and stationarity

$$\theta_{k+1} = \theta_k - H_J(\theta_k)^{-1} \nabla J(\theta_k), \quad H_J(\theta) = \nabla^2 J(\theta).$$

- ▶ Gradient descent uses only the local slope.
- ▶ Newton rescales the slope by curvature, so elongated valleys can be crossed efficiently.
- ▶ Natural gradients use Fisher geometry; they do not generally invert the reward Hessian.



Safety implication

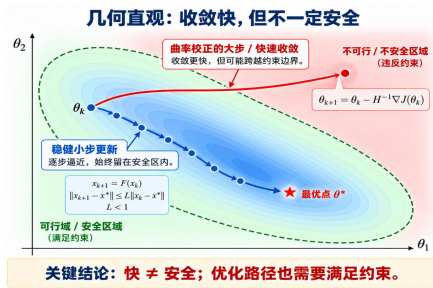
A large curvature-corrected step can be excellent for utility and still leave the feasible region. Curvature does not equal safety.

Newton's step from a Taylor expansion

$$\begin{aligned}\nabla J(\theta + \Delta) &= \nabla J(\theta) + H_J(\theta)\Delta + O(\|\Delta\|^2), \\ 0 &\approx \nabla J(\theta) + H_J(\theta)\Delta.\end{aligned}$$

$$\Delta_N = -H_J(\theta)^{-1}\nabla J(\theta), \quad \theta^+ = \theta + \Delta_N.$$

$$\|\theta^+ - \theta_*\| = O(\|\theta - \theta_*\|^2) \quad \text{locally: Lipschitz Hessian, nonsingular at the root.}$$



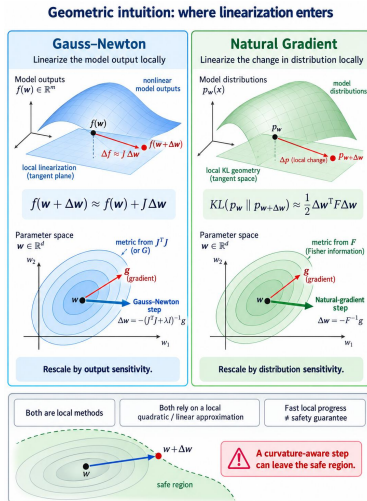
Gauss–Newton and natural-gradient intuition

$$G(\mathbf{w}) = \mathbb{E} [\nabla f(\mathbf{w}) \nabla f(\mathbf{w})^T] .$$

Tikhonov regularization $\Delta_{GN} = -(G + \lambda I)^{-1} g,$

Riemannian Metric $\Delta_{NG} = -F(\mathbf{w})^{-1} g.$

- ▶ Gauss–Newton rescales by output sensitivity.
- ▶ Natural gradient rescales by distributional sensitivity.
- ▶ Both are local coordinate changes, not safety guarantees.



Trust regions as local validity conditions

$$\max_{\Delta} \quad g^{\top} \Delta \quad \text{s.t.} \quad \frac{1}{2} \Delta^{\top} F \Delta \leq \delta.$$

$$\Delta^* = \sqrt{\frac{2\delta}{g^{\top} F^{-1} g}} F^{-1} g.$$

Key idea

Trust-region radius limits the local approximation error. A safety constraint additionally limits the feasible state-policy change.

Function approximation and safety

$$\hat{J}_C(\mathbf{w}) = \mathbb{E}_{d^\pi} [\hat{c}_\mathbf{w}(S, A)], \quad \hat{J}_C(\mathbf{w}) \leq d.$$

approximation error $\Rightarrow$ constraint estimation error,
distribution shift $\Rightarrow$ unseen unsafe regions.

Key idea

A projected value fixed point can be stable while a safety critic is systematically optimistic.

Chapter 8 takeaway

function approximation = parameter optimization around a projected fixed point

- ▶ Linear features make the mean field explicit.
- ▶ Semi-gradients explain why the update is not ordinary gradient descent.
- ▶ Curvature and trust regions control local geometry; safety needs an additional feasible-set argument.

Why the next chapter follows. Multi-step targets remain expensive to implement online. Eligibility traces convert their forward definition into a backward recursion.

Chapter 9 | Eligibility Traces

desired target: $G_t^\lambda = (1 - \lambda) \sum_{n \geq 1} \lambda^{n-1} G_{t:t+n},$

implementation problem: $G_{t:t+n}$ arrives only after future rewards,

backward recursion: $\mathbf{e}_t = \gamma \lambda \mathbf{e}_{t-1} + \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t).$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \delta_t \mathbf{e}_t.$$

Key idea

The trace stores exactly the temporal weights required by the forward mixture.

Forward view: the lambda return

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}, \quad 0 \leq \lambda < 1.$$

$$G_t^\lambda - V(S_t) = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} [G_{t:t+n} - V(S_t)].$$

$$\lambda = 0 \Rightarrow \text{TD}(0), \quad \lambda \rightarrow 1 \Rightarrow \text{MC}.$$

Backward view: eligibility accumulation

$$\mathbf{e}_t = \gamma\lambda\mathbf{e}_{t-1} + \mathbf{x}_t.$$

$$\begin{aligned}\mathbf{e}_t &= \mathbf{x}_t + \gamma\lambda\mathbf{x}_{t-1} + (\gamma\lambda)^2\mathbf{x}_{t-2} + \cdots, \\ \Delta\mathbf{w}_t &= \alpha\delta_t[\mathbf{x}_t + \gamma\lambda\mathbf{x}_{t-1} + \cdots].\end{aligned}$$

Key idea

A current TD error is broadcast backward with exponentially decaying eligibility.

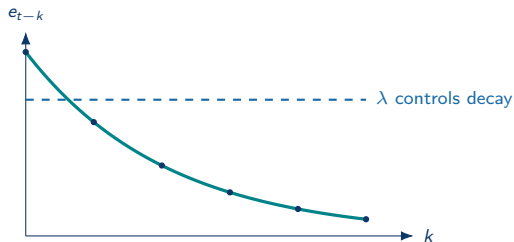
Forward–backward equivalence: key algebra

$$\begin{aligned}\sum_t \alpha \delta_t \mathbf{e}_t &= \sum_t \alpha \delta_t \sum_{k=0}^t (\gamma \lambda)^{t-k} \mathbf{x}_k \\ &= \sum_k \alpha \mathbf{x}_k \sum_{t=k}^{\infty} (\gamma \lambda)^{t-k} \delta_t.\end{aligned}$$

backward accumulation over TD errors $\iff$ forward mixture of n-step returns.

Talk: The proof is a change of summation order. The trace stores exactly the weights that the forward view would apply later.

Eligibility as a temporal filter



memory kernel $k \mapsto (\gamma\lambda)^k$.

- ▶ Large λ gives long credit assignment.
- ▶ Large $\gamma\lambda$ gives slow decay.
- ▶ A trace is a causal low-pass filter over TD errors.

Bias–variance of lambda

$$G_t^\lambda = (1 - \lambda)G_{t:t+1} + \lambda(1 - \lambda)G_{t:t+2} + \cdots .$$

$\lambda \downarrow \Rightarrow$ more bootstrapping, lower variance,

$\lambda \uparrow \Rightarrow$ more sampled return, lower bootstrap bias.

λ tunes temporal credit, not only statistical variance.

Trace dynamics under policy change

$$\mathbf{e}_t = \gamma\lambda\mathbf{e}_{t-1} + \nabla\hat{v}(S_t, \mathbf{w}_t), \quad \mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\delta_t\mathbf{e}_t.$$

- ▶ If the policy changes faster than traces decay, old credit points to a stale distribution.
- ▶ Safe evolution can require trace reset when a policy update crosses a trust-region boundary.

Chapter 9 takeaway

eligibility traces = multi-step credit assignment implemented online

- ▶ Forward view defines the target mixture.
- ▶ Backward view defines the efficient recursion.
- ▶ Safety requires attention to stale eligibility, off-policy ratios, and update resets.

Why the next chapter follows. Estimator design does not define which policies are admissible. CMDPs add an explicit feasible set.

Chapter 10 | Safety Constraints

ordinary RL: $\max_{\pi} J_R(\pi),$

missing information: J_R does not state which policies are unacceptable,

CMDP: $\max_{\pi} J_R(\pi) \quad \text{s.t.} \quad J_{C_i}(\pi) \leq d_i, \quad i = 1, \dots, m.$

$$\Pi_{\text{feas}} = \{\pi : J_{C_i}(\pi) \leq d_i \forall i\}.$$

Key idea

Reward selects among feasible policies. Cost constraints define feasibility.

Discounted occupancy measure

$$x_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \mathbf{1}\{S_t = s, A_t = a\} \right].$$

$$\sum_{s,a} x_{\pi}(s, a) = \frac{1}{1 - \gamma},$$

$$J_R(\pi) = \sum_{s,a} x_{\pi}(s, a) r(s, a),$$

$$\pi(a \mid s) = \frac{x_{\pi}(s, a)}{\sum_b x_{\pi}(s, b)} \quad \text{when } \sum_b x_{\pi}(s, b) > 0.$$

Key idea

Occupancy is the optimization variable. At an unvisited state, the recovered policy may be arbitrary.

Flow conservation

$$x_{\pi}(s, a) = \mu(s)\pi(a|s) + \gamma \sum_{s', a'} P(s|s', a') x_{\pi}(s', a') \pi(a|s)$$

$$\sum_a x_{\pi}(s, a) = \sum_a \mu(s)\pi(a|s) + \gamma \sum_a \sum_{s', a'} P(s|s', a') x_{\pi}(s', a') \pi(a|s)$$

$$\sum_a x_{\pi}(s, a) = \mu(s) + \gamma \sum_{s', a'} P(s|s', a') x_{\pi}(s', a')$$

$$\sum_a x(s, a) - \gamma \sum_{s', a'} P(s | s', a') x(s', a') = \mu(s).$$

left side = discounted mass leaving s – discounted mass entering s ,
right side = initial mass injected at s .

Key idea

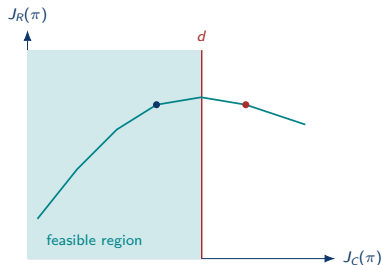
This equality is the discounted analogue of mass conservation in a network flow.

CMDP as a linear program

$$\begin{aligned} \max_{x \geq 0} \quad & \sum_{s,a} x(s,a) r(s,a) \\ \text{s.t.} \quad & Ax = \mu, \\ & \sum_{s,a} x(s,a) c_i(s,a) \leq d_i. \end{aligned}$$

- ▶ Objective: expected discounted reward.
- ▶ Equality constraints: Markov flow.
- ▶ Inequalities: safety budgets.

Reward-cost geometry



$$\pi^* = \arg \max_{\pi \in \Pi_{\text{feas}}} J_R(\pi).$$

Key idea

The unconstrained optimum can be outside the feasible region; the constrained optimum lies on the boundary when safety is active.

Lagrangian relaxation

$$\mathcal{L}(\pi, \lambda) = J_R(\pi) - \lambda^\top (J_C(\pi) - d), \quad \lambda \geq 0.$$

$$\min_{\lambda \geq 0} \mathcal{L}(\pi, \lambda) = \begin{cases} J_R(\pi), & J_C(\pi) \leq d, \\ -\infty, & J_C(\pi) > d. \end{cases}$$

$$\max_{\pi} \min_{\lambda \geq 0} \mathcal{L}(\pi, \lambda) = \max_{\pi \in \Pi_{\text{feas}}} J_R(\pi) \quad \text{under the tabular convex formulation.}$$

Find the saddle.

KKT conditions for a CMDP

Assume an interior smooth policy parameterization π_θ .

$$\begin{aligned} J_C(\theta^*) &\leq d, & \lambda^* &\geq 0, \\ \lambda_i^*(J_{C_i}(\theta^*) - d_i) &= 0, \\ \nabla_\theta J_R(\theta^*) - \sum_i \lambda_i^* \nabla_\theta J_{C_i}(\theta^*) &= 0. \end{aligned}$$

A positive multiplier implies an active constraint; the converse need not hold.

For policies on the simplex boundary, use the occupancy-measure LP KKT system, including nonnegativity multipliers.

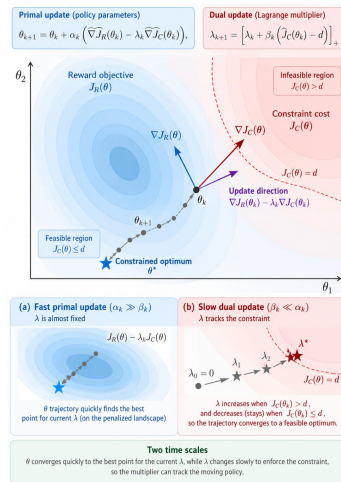
Primal-dual stochastic approximation

$$\theta_{k+1} = \theta_k + \alpha_k \left(\widehat{\nabla J_R(\theta_k)} - \lambda_k \widehat{\nabla J_C(\theta_k)} \right),$$

$$\lambda_{k+1} = \left[\lambda_k + \beta_k \left(\widehat{J_C(\theta_k)} - d \right) \right]_+.$$

$$\alpha_k \ll \beta_k \quad \text{or} \quad \beta_k \ll \alpha_k$$

- Two time scales can let the multiplier track a moving policy.



Why fixed penalties are not constraints

$$\max_{\pi} J_R(\pi) - \lambda J_C(\pi)$$

λ fixed $\Rightarrow$ one point on a reward–cost tradeoff curve,

λ adaptive $\Rightarrow$ attempt to enforce $J_C(\pi) \leq d$.

Key idea

A fixed penalty can select an infeasible policy when the scale or curvature of the cost changes.

Trust-region constrained policy improvement

$$\max_{\theta} \hat{J}_R(\theta; \theta_k) \quad \text{s.t.} \quad \hat{J}_C(\theta; \theta_k) \leq d, \quad D_{KL}(\pi_{\theta_k} \| \pi_{\theta}) \leq \delta.$$

- ▶ The surrogate objective uses samples from the old policy.
- ▶ The trust region controls distribution shift.
- ▶ The cost constraint controls the local safety direction.

CPO-style local quadratic program

$$\begin{aligned} \max_x \quad & g^T x \\ \text{s.t.} \quad & b^T x + c \leq 0, \\ & \frac{1}{2} x^T H x \leq \delta. \end{aligned}$$

$$\begin{aligned} g &\approx \nabla_{\theta} J_R, & b &\approx \nabla_{\theta} J_C, \\ H &\approx \text{Fisher information matrix.} \end{aligned}$$

Key idea

CPO converts local policy improvement into constrained second-order optimization.

Original CPO experiment figure

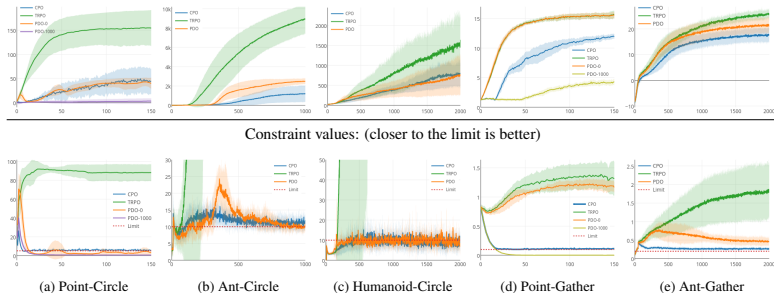
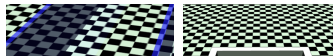


Figure 1. Average performance for CPO, PDO, and TRPO over several seeds (5 in the Point environments, 10 in all others); the x -axis is training iteration. CPO drives the constraint function almost directly to the limit in all experiments, while PDO frequently suffers from over- or under-correction. TRPO is included to verify that optimal unconstrained behaviors are infeasible for the constrained problem.

- **Circle:** The agent is rewarded for running in a wide circle, but is constrained to stay within a safe region smaller than the radius of the target circle



Constraint satisfaction during learning

terminal feasibility $\neq$ trajectory-wise feasibility during updates.

$$\forall k: \quad J_C(\pi_k) \leq d$$

$$J_C(\pi_K) \leq d$$

strong training-time guarantee,

weak terminal guarantee.

Key idea

For evolving agents, unsafe intermediate policies can damage the system before the final policy is reached.

Safe evolution as a CMDP over updates

$$M_{t+1} = \mathcal{E}(M_t, \tau_t; \theta_t).$$

$$\begin{aligned} \max_{\theta} \quad & U(\theta) + \beta P_{u \rightarrow s}(\theta) \\ \text{s.t.} \quad & P_{s \rightarrow u}(\theta) \leq \varepsilon. \end{aligned}$$

optimization variable = policy update / memory update / skill update.

Chapter 10 takeaway

CMDP = utility optimization over a constrained occupancy geometry

- ▶ LP: exact convex geometry for finite tabular CMDPs.
- ▶ Lagrangian: dual prices for constraint violations.
- ▶ Trust region: local control of distribution shift.
- ▶ Safe evolution: apply the same logic to agent updates themselves.

Why the next chapter follows. An expected-cost constraint still misses tail severity, model error, and violation probability.

Chapter 11 | Better Safety

CMDP controls: $\mathbb{E}_{\pi}[C] \leq d$,

remaining questions: How bad is the tail? Is the model correct? How often is $C > d$?

new formulations: CVaR, $\max_{\pi} \min_{P \in \mathcal{P}} J(\pi, P), \quad \mathbb{P}(C > d) \leq \delta.$

Key idea

The safety formulation must match the failure mode, because expectation, tail risk, model uncertainty, and violation probability are different objects.

Risk-neutral optimization can hide catastrophes

$$\max_{\pi} \mathbb{E}_{\pi}[G]$$

$G_A = 10$ with probability 1,

$G_B = 101$ with probability 0.99, -9899 with probability 0.01.

$$\mathbb{E}[G_A] = 10, \quad \mathbb{E}[G_B] = 1,$$

- ▶ the expectation detects the lower mean;
- ▶ the optimization problem does not encode the severity of the rare event.

Mean–variance objective

Markowitz Portfolio Theory

$$\max_{\pi} \quad \mathbb{E}[G] - \beta \operatorname{Var}(G).$$

$$\operatorname{Var}(G) = \mathbb{E}[G^2] - (\mathbb{E}[G])^2$$

$$\nabla_{\theta} \operatorname{Var}(G) = \nabla_{\theta} \mathbb{E}[G^2] - 2\mathbb{E}[G] \nabla_{\theta} \mathbb{E}[G].$$

risk penalty = second-moment optimization,

Key idea

Mean–variance is interpretable but is not generally time-consistent under dynamic programming.

Exponential utility and risk-sensitive Bellman equations

$$\mathcal{E}_\eta(X) = \frac{1}{\eta} \log \mathbb{E}[e^{\eta X}], \quad \eta \neq 0.$$

$$V_t(s) = \max_a \frac{1}{\eta} \log \sum_{s', r} p(s', r \mid s, a) \exp(\eta[r + \gamma V_{t+1}(s')]).$$

- ▶ $\eta < 0$ penalizes downside dispersion for rewards; $\eta > 0$ emphasizes upside.
- ▶ This Bellman equation defines a recursive entropic criterion.

A static utility of the discounted total return generally needs an augmented state; it is not obtained by replacing continuation with $V(s')^\gamma$.

CVaR as a tail objective

Rockafellar-Uryasev

$$\text{CVaR}_\alpha(L) = \min_{\zeta \in \mathbb{R}} \left[\zeta + \frac{1}{1-\alpha} \mathbb{E}[(L - \zeta)_+] \right].$$

$$\begin{aligned} \text{VaR}_\alpha(L) &= \inf\{\zeta : F_L(\zeta) \geq \alpha\}, \\ \text{CVaR}_\alpha(L) &= \mathbb{E}[L \mid L \geq \text{VaR}_\alpha(L)] \quad \text{for continuous } L. \end{aligned}$$

Key idea

CVaR optimizes the average loss in the worst $1 - \alpha$ tail.

Robust MDP objective

$$\max_{\pi} \min_{P \in \mathcal{P}} J(\pi, P).$$

$P \in \mathcal{P}$ models transition uncertainty,

$$\pi^* = \arg \max_{\pi} \inf_{P \in \mathcal{P}} J(\pi, P).$$

- ▶ The inner minimizer is an adversarial model.
- ▶ Robustness optimizes worst-case performance, not estimated average performance.

Robust Bellman operator

$$(T_{\text{rob}} V)(s) = \max_a \min_{p \in \mathcal{P}(s,a)} \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) V(s') \right].$$

$$\|T_{\text{rob}} V - T_{\text{rob}} U\|_{\infty} \leq \gamma \|V - U\|_{\infty}.$$

Key idea

Rectangular uncertainty preserves a Bellman-style contraction. Nonrectangular uncertainty can destroy dynamic consistency.

Chance constraints

$$\mathbb{P}_\pi \left(\sum_{t=0}^{T-1} C_t > d \right) \leq \delta.$$

$$\mathbb{E}[C] \leq d$$

controls average cost,

$$\mathbb{P}(C > d) \leq \delta$$

controls violation frequency.

$$\mathbb{P}(C > d) \leq \frac{\mathbb{E}[C]}{d} \quad \text{for } C \geq 0, d > 0; \text{ Markov's bound is often loose.}$$

From chance constraints to CVaR bounds

$$\text{CVaR}_{1-\delta}(C) \leq d \implies \mathbb{P}(C > d) \leq \delta.$$

- ▶ CVaR can serve as a convex sufficient condition for a chance constraint.
- ▶ The implication is conservative: a chance constraint may hold while the CVaR bound fails.
- ▶ Safety level is now a tail quantile parameter.

Safe exploration as constrained information gathering

$$\max_{\pi} \text{information gain}(\pi) \quad \text{s.t.} \quad \mathbb{P}_{\pi}(\text{unsafe state}) \leq \delta.$$

unknown action $\neq$ safe action,
exploration value = future information – current risk.

Key idea

Safe exploration turns uncertainty into a constraint, not merely an optimism bonus.

Reachability and safe sets

$$\mathcal{S}_{\text{safe}} = \{s : V_C(s) \leq d\}.$$

$$\mathcal{K}_0 = \mathcal{S}_{\text{safe}},$$

$$\mathcal{K}_{k+1} = \{s : \exists a, \text{ supp } P(\cdot \mid s, a) \subseteq \mathcal{K}_k\}.$$

$$\mathcal{K}_\infty = \bigcap_{k \geq 0} \mathcal{K}_k \text{ is a controlled invariant safe kernel.}$$

Chapter 11 takeaway

safety is a choice of risk functional, model uncertainty, or event constraint

- ▶ Risk-sensitive: change the objective over returns.
- ▶ Robust: optimize the worst model in an uncertainty set.
- ▶ Chance-constrained: control violation probability.
- ▶ Safe exploration: constrain information-gathering actions.

Why the next chapter follows. Modern algorithms combine these formulations with a small set of numerical primitives.

Chapter 12 | Algorithm names reduce to optimization primitives

common ingredients: π_θ , $J_R(\theta)$, $J_C(\theta)$,
design choice: penalty, multiplier, projection, trust region, barrier, or shield,
causal effect: primitive $\longrightarrow$ update geometry $\longrightarrow$ safety claim.

Key idea

To understand a method, identify what changes, what is constrained, and when its guarantee applies.

Primitive 1: reward shaping

$$r'(s, a) = r(s, a) - \lambda c(s, a), \quad \max_{\pi} J_{r'}(\pi).$$

λ fixed $\Rightarrow$ unconstrained scalarization,

λ tuned $\Rightarrow$ scale-sensitive feasibility.

$\arg \max J_R - \lambda J_C$ need not satisfy $J_C \leq d$.

Primitive 2: primal–dual optimization

$$\max_{\theta} \min_{\lambda \geq 0} J_R(\theta) - \lambda(J_C(\theta) - d).$$

$$\begin{aligned}\theta_{k+1} &= \theta_k + \alpha_k(\nabla J_R - \lambda_k \nabla J_C), \\ \lambda_{k+1} &= [\lambda_k + \beta_k(J_C - d)]_+.\end{aligned}$$

- ▶ New variable: multiplier.
- ▶ New risk: dual oscillation and constraint lag.

Primitive 3: projection

$$\theta' = \theta + \Delta, \quad \theta_{\text{safe}} = \Pi_{\mathcal{F}}(\theta') = \arg \min_{z \in \mathcal{F}} \|z - \theta'\|^2.$$

unconstrained improvement $\rightarrow$ projection onto feasible set,

$$\mathcal{F} = \{\theta : J_C(\theta) \leq d\}.$$

Key idea

Projection preserves feasibility only if $\mathcal{F}$ is known or conservatively approximated.

Primitive 4: trust regions

$$\max_{\theta} \hat{J}_R(\theta) \quad \text{s.t.} \quad D_{KL}(\pi_{\theta_k} \parallel \pi_{\theta}) \leq \delta.$$

local guarantee $\Rightarrow$ bounded distribution shift,
not automatically $\Rightarrow$ safe cost.

trust region = stability constraint, not a safety constraint.

Primitive 5: barrier methods

$$\max_{\theta} \quad J_R(\theta) + \mu \log(d - J_C(\theta)), \quad J_C(\theta) < d.$$

$$\nabla_{\theta} \mathcal{J} = \nabla J_R(\theta) - \frac{\mu}{d - J_C(\theta)} \nabla J_C(\theta).$$

$$J_C \uparrow d \quad \Rightarrow \quad \text{barrier gradient} \uparrow.$$

Primitive 6: risk-sensitive objectives

$$\min_{\pi} \text{CVaR}_{\alpha}(L_{\pi}), \quad \max_{\pi} \mathbb{E}[G] - \beta \text{Var}(G).$$

- ▶ The feasible set may be unchanged.
- ▶ The optimizer changes which trajectories dominate the gradient.
- ▶ Risk sensitivity protects tails but does not necessarily enforce a hard budget.

Primitive 7: robust min–max optimization

$$\max_{\pi} \min_{M \in \mathcal{M}} J(\pi, M).$$

policy optimizer $\leftrightarrow$ adversarial model optimizer,
robustness = performance under model misspecification.

$\mathcal{M}$ too large $\Rightarrow$ over-conservative behavior.

Primitive 8: shielding and action correction

$$a_{\text{safe}} = \arg \min_{a' \in \mathcal{A}_{\text{safe}}(s)} d(a', a), \quad a \sim \pi_{\theta}(\cdot | s).$$

π_{θ} proposes,
 $\mathcal{S}$ or $\mathcal{A}_{\text{safe}}$ filters.

Key idea

Shielding moves safety from the objective into a runtime admissibility map.

A taxonomy of guarantees

Shaping scalar reward

No feasibility guarantee; penalty calibration can fail.

Lagrangian reward and dual cost

Average or asymptotic feasibility; dual lag can oscillate.

Projection feasible parameter set

Local feasibility; the estimated set may be wrong.

Trust region bounded policy shift

Local stability; the safety constraint may still fail.

Barrier interior objective

Stays inside an estimated boundary; boundary error dominates.

Risk objective tail functional

Controls a tail statistic; it does not impose a hard budget.

Robust objective worst model

Model-wise performance; a large uncertainty set is conservative.

Shield admissible action

Runtime action safety; the shield may be incomplete.

One safety budget, three update mechanisms

fixed penalty: $\max_{\theta} J_R(\theta) - \lambda J_C(\theta)$
 λ does not react to $J_C - d$,

primal-dual: $\lambda_{k+1} = [\lambda_k + \beta_k(J_C - d)]_+$
violations increase the future penalty,

CPO-style step: $\max_{\Delta} g^T \Delta$ s.t. $b^T \Delta + c \leq 0$
the local step must satisfy a surrogate constraint.

same (J_R, J_C, d) + different feedback = different safety behavior.

How to read a safety claim

claim = (quantity, probability, time scope, model scope).

$$\mathbb{P}(J_C(\pi_k) \leq d \ \forall k) \geq 1 - \delta$$

$$\mathbb{E}[J_C(\pi_K)] \leq d$$

$$J_C(\pi_K) \leq d$$

strong,

weaker,

single-run claim.

Key idea

A safety result is incomplete without its probability and time quantifiers.

Algorithm audit: what is being optimized?

1. Identify the primal variable: Q , V , π , θ , λ , or the agent update.
2. Identify the target: return, Bellman backup, surrogate, risk functional, or worst-case model.
3. Identify the feasible object: actions, policies, occupancies, parameters, or trajectories.
4. Identify where uncertainty enters: target, model, constraint estimate, or exploration rule.

Algorithm audit: what can fail?

failure $\in$ {bias, variance, instability, distribution shift, constraint error}.

unbiased target $\nrightarrow$ low variance,
stable update $\nrightarrow$ safe policy,
safe final policy $\nrightarrow$ safe learning path.

From safe RL to safe evolution

$$\theta_{t+1} = \theta_t + \Delta_t, \text{quad} \Delta_t \in \mathcal{U}(\theta_t),$$

$$\mathcal{U}(\theta) = \{\Delta : \text{utility improves and safety remains feasible}\},$$

$\mathcal{E}(M_t, \tau_t; \theta_t)$ is the dynamics of the update itself.

Key idea

The safe-RL primitive must be lifted from action selection to update selection.

Chapter 12 takeaway

modern safe RL is a library of optimization primitives

- ▶ Ask what variable changes.
- ▶ Ask what set is constrained.
- ▶ Ask whether the guarantee is local, asymptotic, probabilistic, or runtime.
- ▶ Ask how the primitive behaves under self-modification.

Why the next chapter follows. The final chapter recomposes objectives, estimators, updates, and constraints into one model of safe evolution.

Chapter 13 | The unified optimization viewpoint

sequential decisions $\implies v = Tv$,

unknown expectations $\implies$ sample targets and stochastic approximation,

large state spaces $\implies \Pi T$ and parameter optimization,

unsafe optima $\implies \max J_R$ s.t. $J_C \leq d$,

self-modification $\implies M_{t+1} = \mathcal{E}(M_t, \tau_t; \theta_t)$.

Key idea

Each chapter added one mathematical object only when the previous formulation could no longer express the problem.

View 1: algorithms estimate one Bellman object

$$v_{\pi} = T^{\pi} v_{\pi}, \quad v_{*} = T v_{*}.$$

DP : exact operator,

MC : full return sample,

TD : bootstrapped operator sample.

Key idea

The algorithm changes the estimator and update geometry, not the semantic Bellman target.

View 2: target design is a bias–variance choice

$$\text{TD}(0) \leftrightarrow n\text{-step TD} \leftrightarrow \text{TD}(\lambda) \leftrightarrow \text{MC}.$$

short target $\rightarrow$ local, biased, lower variance,

long target $\rightarrow$ delayed, less biased, higher variance.

View 3: safety changes the feasible geometry

$$\max J_R(\pi) \quad \rightsquigarrow \quad \max J_R(\pi) \quad \text{s.t.} \quad J_C(\pi) \leq d.$$

reward shaping : changes scalar objective,

CMDP : changes feasible set,

robust/risk : changes adversary or tail functional.

View 4: safe evolution constrains updates

$$\max_{\theta} U(\theta) + \beta P_{u \rightarrow s}(\theta) \quad \text{s.t.} \quad P_{s \rightarrow u}(\theta) \leq \varepsilon.$$

- ▶ The optimization variable can be memory, skill, policy, architecture, or update rule.
- ▶ The safe set is a set of agent states, not only a set of actions.
- ▶ A safe endpoint is insufficient if the evolution path is unsafe.

Open questions

1. How can we define a feasible update region for memory and skill evolution?
2. Can we guarantee safe $\rightarrow$ unsafe transitions are rare while allowing unsafe $\rightarrow$ safe repair?
3. Which contraction, Lyapunov, or trust-region arguments survive a changing agent architecture?
4. Can we estimate tail safety under distribution shift without freezing evolution?

Safe agent evolution is sequential constrained optimization over agent updates.

optimize the update while proving the update remains admissible



Xianzhe Meng

Thank you

References I

- Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning*, pages 22–31, 2017.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016.
- Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. In *Machine Learning*, volume 47, pages 235–256, 2002.
- Richard Bellman. Dynamic programming. *Princeton University Press*, 1957.
- Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.
- Yinlam Chow, Aviv Tamar, Shie Mannor, and Marco Pavone. Risk-sensitive and robust decision-making: A cvar optimization approach. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- Tom Everitt, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective. *Synthese*, 198:6435–6467, 2021.
- Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(42):1437–1480, 2015.
- Garud N. Iyengar. Robust dynamic programming. *Mathematics of Operations Research*, 30(2):257–280, 2005.
- Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6:4–22, 1985.

References II

- Teodor M. Moldovan and Pieter Abbeel. Safe exploration in markov decision processes. *arXiv preprint arXiv:1205.4810*, 2012.
- Arnab Nilim and Laurent El Ghaoui. Robust control of markov decision processes with uncertain transition matrices. *Operations Research*, 53(5):780–798, 2005.
- Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley and Sons, 1994.
- Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.
- Satinder P. Singh, Tommi Jaakkola, Michael L. Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine Learning*, 38:287–308, 2000.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018. URL <http://incompleteideas.net/book/the-book-2nd.html>.
- Aviv Tamar, Yonatan Glassner, and Shie Mannor. Optimizing the cvar via sampling. *arXiv preprint arXiv:1404.3862*, 2015.
- John N. Tsitsiklis. Asynchronous stochastic approximation and q-learning. *Machine Learning*, 16:185–202, 1994.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992. doi: 10.1007/BF00992698.